Speech and Vision Integrated Robotic Arm for Smart Picking

A project report submitted in partial fulfilment of the requirements for the degree of

Bachelor of Technology

in

Electronics and Computer Engineering

by

Revan MJ

(22BLC1345)

Velmurugan S

(22BLC1403)

Sham Ganesh M

(22BLC1341)

School of Electronics Engineering,
Vellore Institute of Technology, Chennai,
Vandalur–Kelambakkam Road,
Chennai – 600127, India.

April 2026

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

CHENNAI

School of Electronics Engineering

Declaration

We hereby declare that the report titled ***"Speech and Vision Integrated Robotic Arm for Smart picking"*** submitted by us to the School of Electronics Engineering, Vellore Institute of Technology, Chennai in partial fulfillment of the requirements for the award of **Bachelor of Technology** in **Electronics and Computer Engineering** is a bonafide record of the work carried out by us under the supervision of ***Dr. Idayachandran***.

We further declare that the work reported in this report, has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma of this institute or of any other institute or University.

Signature: Revan.MJ, [signature], [signature]

Name & Reg. No.: REVAN MJ, SHAM GANESH M, VELMURUGAN S
22BLC1345 22BLC1341 22BLC1403

Date: 08/04/2026

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

CHENNAI

School of Electronics Engineering

Certificate

This is to certify that the project report titled ***Speech and Vision Integrated Robotic Arm for Smart Picking*** submitted by **Revan MJ (22BLC1345), Velmurugan S (22BLC1403) and Sham Ganesh M (22BLC1341)** to Vellore Institute of Technology Chennai, in partial fulfillment of the requirement for the award of the degree of **Bachelor of Technology** in **Electronics and Computer Engineering** is a bonafide work carried out under my supervision. The project report fulfills the requirements as per the regulations of this University and in my opinion meets the necessary standards for submission. The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma and the same is certified.

Supervisor	**Head of the Department**
Signature:	Signature: .....
Name: JAYACHANDRAN-G	Name: DR. VIJAYAKUMAR - P
Date: 08/04/2026	Date: 08-04-2026

Head of the Department
B.Tech.(ECM) Programme
School of Electronics Engineering
Vellore Institute of Technology(VIT)
Vandalur - Kelambakkam Road,
Chennai - 600 127.

Examiner 1	**Examiner 2**
Signature:	Signature: .....
Name:	Name: .....
Date:	Date: .....

(Seal of the School)

Vellore Institute of Technology
School of Electronics Engineering
Chennai - 600127, Tamil Nadu, India

Abstract

Traditional human-robot interaction (HRI) presents a significant barrier to the widespread adoption of robotics in everyday life. Handling robotic systems is always tricky, especially in dynamic, human-centric environments, as complex programming and joystick-based controls are non-intuitive, limiting their use to trained experts. Integrating multiple sensory modalities, like speech and vision, to make a robot understand and act on complex, non-repetitive tasks has always been a challenge. The complexity further increases when the system must perceive, reason, and act in real-time.

Existing robotic systems often work perfectly with pre-defined, keyword-based commands or in environments with known objects. However, when analysis of complex, natural language instructions is required, these methods do not suit our need. This is mainly because they lack a deep, contextual bridge between the user's linguistic intent and the robot's visual perception of the world.

Instead, in this paper, we propose a multimodal control architecture that bridges this interaction gap. Our system integrates a state-of-the-art speech-to-text engine (whisper.cpp) with a lightweight on-device large language model (Qwen2 1.5B GGUF) for robust, zero-shot intent extraction. This extracted intent — such as "find the blue pen" — is then used to guide a YOLO11s-seg vision pipeline, which performs real-time instance segmentation to localise the target object. The entire pipeline runs natively on an NVIDIA Jetson Orin Nano, without any cloud dependency. By decoupling the complexities of multi-modal AI from the low-level hardware control — handled by a dedicated Arduino over Serial — we present a scalable and intuitive system that empowers non-expert users to interact with robotic arms in unstructured, real-time environments.

Acknowledgements

We wish to express our sincere thanks and deep sense of gratitude to our project guide, **Dr. Idayachandran**, Professor, School of Electronics Engineering, for his consistent encouragement and valuable guidance offered to us in a pleasant manner throughout the course of the project work.

We are extremely grateful to Dean Dr. Ravi Sankar A, Associate Dean (Academics) Dr. Mohana Prasad K, & Associate Dean (Research) Dr. Chitra K, of the School of Electronics Engineering, VIT Chennai, for extending the facilities of the school towards our project and for his unstinting support.

We express our thanks to our Head of the Department Dr. Vijayakumar P for his support throughout the course of this project.

We also take this opportunity to thank all the faculty of the School for their support and their wisdom imparted to us throughout the course.

We thank our parents, family, and friends for bearing with us throughout the course of our project and for the opportunity they provided us in undergoing this course in such a prestigious institution.

Codes and Standards

Project Title: *Speech and Vision Integrated Robotic Arm for Smart Picking*

1. **Robotics & Safety Standards**
   - **ISO 10218 – Robots and Robotic Devices: Safety**
     Ensures the robotic arm operates safely in proximity to humans.
   - **IEC 61508 – Functional Safety of E/E/PE Systems**
     Ensures the embedded control system meets safety integrity levels.
2. **Communication Standards**
   - **IEEE 802.11 – Wi-Fi Communication**
     For transmitting data between the Raspberry Pi and networked modules.
3. **AI & Software Standards**
   - **ISO/IEC 25010 – Software Quality**
     Guides reliability, performance efficiency, and maintainability attributes.
4. **Electrical Safety**
   - **IEC 60364 – Electrical Installations Safety**
     Ensures safe deployment of servo drivers and power supplies.
   - **IEC 62133 – Battery Safety**
     For safe use of rechargeable power sources.

Mapping of Sustainable Development Goals (SDGs)

Project Title: *Speech and Vision Integrated Robotic Arm for Smart Picking*

Project Title	Mapped SDGs	Justification	Example Impact
Speech and Vision Integrated Robotic Arm for Smart Picking	**SDG 9** – Industry, Innovation and Infrastructure	Uses state-of-the-art AI (LLMs, vision models) for smart automation.	Promotes intelligent manufacturing and accessible robotic systems.
	SDG 10 – Reduced Inequalities	Voice and vision control removes need for expert programming, democratising robot use.	Enables physically challenged individuals to use robotic assistants independently.
	SDG 3 – Good Health and Well-being	Can assist elderly or disabled users with picking and placing objects.	Improves daily independence and quality of life for users with mobility challenges.
	SDG 4 – Quality Education	Demonstrates integration of AI, robotics, and NLP for educational advancement.	Serves as a teaching platform for next-generation robotics curricula.

Contents

Declaration i

Certificate ii

Abstract iii

Acknowledgements iv

Codes and Standards v

Mapping of Sustainable Development Goals (SDGs) vi

List of Acronyms xii

1 Introduction 1

2 Literature Survey 3

2.1 HMM-Based Phoneme Speech Recognition System for Control and Command of Industrial Robots 3

2.2 Control of Robot Arm Based on Speech Recognition Using MFCC and KNN Method 3

2.3 Robotic Arm Control Using Gesture and Voice 4

2.4 Wireless Voice Controlled Robotics Arm 4

2.5 Pick and Place Control of a 3-DOF Robot Manipulator Based on Image and Pattern Recognition 5

2.6 Implementation of Pick and Place Robotic Arm Using Speech Processing and IoT 5

2.7 Voice Controlled Object Grasping Robotic Arm for Visually Impaired Disabled Veterans 6

2.8 Voice Controlled Camera-Assisted Pick and Place Robot Using Raspberry Pi 6

2.9 Smart Autonomous Wireless Gripper for Human-Robot Collaborative Operation 6

2.10 Multimodal Human-Robot Interaction Using Gestures and Speech: A Case Study for Printed Circuit Board Manufacturing 7

2.11 Robotic Arm for Pick and Place Application 7

2.12 Voice-Controlled Pick and Place Robot 8

3 Methodology **9**

3.1 The Software and Hardware Stack 10

3.1.1 Hardware Configuration 10

3.1.2 Software Stack 10

3.2 Layer 1: Perception 11

3.2.1 Audio and NLU 11

3.2.2 Vision 12

3.3 Layer 2: Planning and Execution 12

4 Results and Discussions **14**

4.1 Objective 14

4.2 NLU Pipeline: whisper.cpp and Qwen2 1.5B (GGUF) 14

4.3 Vision Pipeline: YOLO11s-seg 15

4.3.1 YOLO11s-seg: Instance Segmentation for Object Localisation 15

4.4 Hardware Setup 17

5 Conclusion and Future Scope **20**

6 Appendix **22**

6.1 master.cpp 22

6.2 play_rec.cpp 24

6.3 rec_till_r.cpp 25

6.4 whisp_embed_nu.cpp 27

6.5 whisper_call.cpp 29

6.6 whisper_call_log.cpp 30

6.7 Intent Client Server Code 32

6.8 Qwen + Whisper Pipeline Code 34

6.9 FK / IK Solver Code 36

6.10 Test Angle Control Code 40

6.11 YOLO Object Detection Code 42

6.12 Servo Control Code 45

6.13 TOF Sensor Code 49

Bibliography **50**

Biodata **51**

List of Figures

3.1 Architecture 9

4.1 Qwen2 1.5B (GGUF) Test Score 15

4.2 Whisper Speech-to-Text Translation 16

4.3 Qwen2 1.5B (GGUF) JSON Outputs for sample inputs 16

4.4 YOLO11s-seg performing instance segmentation on the scene 17

4.5 Top View of the Hardware Setup 18

4.6 Side View of the Hardware Setup 19

6.1 master.cpp – Part 1 22

6.2 master.cpp – Part 2 23

6.3 play_rec.cpp 24

6.4 rec_till_r.cpp – Part 1 25

6.5 rec_till_r.cpp – Part 2 26

6.6 whisp_embed_nu.cpp – Part 1 27

6.7 whisp_embed_nu.cpp – Part 2 28

6.8 whisper_call.cpp 29

6.9 whisper_call_log.cpp – Part 1 30

6.10 whisper_call_log.cpp – Part 2 31

6.11 Intent Client Server Code – Part 1 32

6.12 Intent Client Server Code – Part 2 33

6.13 Qwen + Whisper Pipeline Code – Part 1 34

6.14 Qwen + Whisper Pipeline Code – Part 2 35

6.15 FK/IK Code – Part 1 36

6.16 FK/IK Code – Part 2 37

6.17 FK/IK Code – Part 3 38

6.18 FK/IK Code – Part 4 39

6.19 FK/IK Code – Part 5 39

6.20 Test Angle Control Code – Part 1 40

6.21 Test Angle Control Code – Part 2 41

6.22 YOLO Object Detection Code – Part 1 42

6.23 YOLO Object Detection Code – Part 2 43

6.24 YOLO Object Detection Code – Part 3 44

6.25 YOLO Object Detection Code – Part 4 44

6.26 Servo Control Code – Part 1 45

6.27 Servo Control Code – Part 2 46

6.28 Servo Control Code – Part 3 47

6.29 Servo Control Code – Part 4 48

6.30 TOF Sensor Code 49

List of Acronyms

HRI	Human-Robot Interaction
NLU	Natural Language Understanding
LLM	Large Language Model
STT	Speech-to-Text
IK	Inverse Kinematics
DOF	Degrees of Freedom
GPU	Graphics Processing Unit
FPS	Frames Per Second
JSON	JavaScript Object Notation
YOLO	You Only Look Once
TOF	Time-of-Flight
IoT	Internet of Things
MFCC	Mel-Frequency Cepstral Coefficients
KNN	K-Nearest Neighbours
NLP	Natural Language Processing

Chapter 1

Introduction

Robotic capabilities have improved over the years, but their deployment into dynamic environments lags behind. The main issue being the interaction and not the mechanical capabilities. The existing methods to control robots often rely on complex, task-specific programming or using manual controllers, this creates a significant gap in Human-Robot Interaction (HRI). This complexity creates a barrier which prevents non-expert users from using automation in everyday tasks and dynamic environments.

The conventional paradigm of robots inherently has a "legacy rigidity." That is, most systems are programmed to handle a fixed set of operations and are ill-suited for scalability, as well as adaptation to new or changing requirements. This further contributes to the "interaction gap": the unreliability of existing multimodal systems in real-world applications. As much as robots have been equipped with sensors, they have traditionally struggled to generalise learned behaviour. Often, minor variations in their environment call for exhaustive retraining.

The present project addresses this adaptability challenge directly. For practical robotics of the future, there will have to be a shift from pre-programmed systems to "Cognitive Embodied AI". This new generation of robot systems should be able to perceive its environment, reason about the world it finds itself in, and autonomously adapt its actions, embedded in dynamic human contexts.

Our solution is to integrate computer vision and natural language processing to bridge this HRI gap. We propose a system that moves beyond simple, keyword-based triggers to embrace the full spectrum of human communication. By leveraging on-device AI — whisper.cpp for speech transcription, Qwen2 1.5B (GGUF) for intent extraction, and YOLO11s-seg for real-time instance segmentation — all running on the NVIDIA Jetson Orin Nano, we aim to build a robot that can be tasked through natural, real-time voice

commands without any cloud dependency.

Chapter 2

Literature Survey

2.1 HMM-Based Phoneme Speech Recognition System for Control and Command of Industrial Robots

This paper actively uses speech recognition technology as interface for human-robot interaction. They mainly focus on industrial robots. The authors mention the challenge of achieving smooth and accurate communication between humans and robots using common control systems like computer links, which results in reduced precision and less adaptability. To solve this, they propose a speech recognition model using Linear Predictive Coding and Hidden Markov Models for identifying commands. The system divides the task into training and recognition, uses an audio data to train the robot to respond to eight basic control commands. The results demonstrate tasks such as pick and place operations with good precision and reduced manual effort. However, this work also has limitations in background noise handling and adaptability to different speech accents. We can possibly integrate with advanced noise-filtering algorithms to improve overall performance.

2.2 Control of Robot Arm Based on Speech Recognition Using MFCC and KNN Method

This work sets up a voice-controlled robot arm with five movements, run by an Arduino chip. Instead of hands-on operation, it listens to speech to act — aimed at helping people with mobility challenges. To capture the command, it pulls out sound traits using MFCCs — a way to map how voices differ. Then, sorting those sounds into commands happens through KNN, which matches new inputs to past examples. By

teaming these two methods, the system responds fast without needing heavy processing power. Tests showed solid results: when familiar users spoke, it got things right 85 times out of 100; even strangers were understood about 4 out of every 5 tries. This shows the system holds up well plus works smoothly for users. Still, it struggles when there's lots of background sound — so next steps might look into live noise cleanup or smart voice sorting using neural networks to make it more dependable.

2.3 Robotic Arm Control Using Gesture and Voice

In this study, the researchers attempted to create a more intuitive human-robot interaction, especially for helping the elderly and disabled people, by integrating hand gesture recognition with voice command inputs to achieve an advanced human-machine interface. This study involved the use of a data glove with a 3-axis accelerometer and PIC18F2510 microcontroller, which detected the position and movement of the hand and sent them wirelessly to the robot arm through an RF 2.4 GHz module. In addition, the speech recognition module, employing Kalman filtering, was developed to permit voice inputs as an alternative method of controlling the arm. The results showed that hand orientations, which were measured in terms of X, Y, and Z deflections, can be matched with certain arm movements. Moreover, voice command execution was proven through a graphical user interface (GUI), thus indicating that human-robot interaction through a dual-mode controller can be done effectively. On the other hand, some limitations were observed in this study, such as the need for speaker-specific voice recognition and the limited gesture recognition capability of the system.

2.4 Wireless Voice Controlled Robotics Arm

This paper aimed to develop a wireless, voice-operated robotic arm that could remove the dependence on manual programming by letting users control arm movement and platform navigation entirely through spoken English commands. The system was built around a PIC microcontroller as the central processing unit, a voice recognition module (using Hidden Markov Models as the core speech processing technique), and a ZigBee (CC2500) transceiver that wirelessly transmitted 4-bit encoded command data from the user's end to the robot receiver over a range of 10 to 50 meters; servo motors controlled the arm's pick, drop, raise, and lower actions, while stepper motors handled platform movement (forward, backward, left, right). The system successfully executed all defined voice commands and demonstrated a functional multi-part robot — arm, gripper, and mobile platform — all working together in a coordinated manner under

voice control. The main problems were that the speech recognition was limited to a predefined, isolated-word vocabulary in English with no support for continuous speech or non-speech sound recognition, and the system didn't have any feedback mechanism when commands were vague or poorly recognized, though the authors flagged these as areas for future development.

2.5 Pick and Place Control of a 3-DOF Robot Manipulator Based on Image and Pattern Recognition

This project introduces an affordable semi-automatic robot setup built for picking and moving items using image and pattern understanding. It shows the difficulties in linking people with robots via vision or speech during live operation. A 3-joint MeArm arm is applied here — fitted with cloud-powered voice detection from Microsoft — to let users shift chess figures by talking. The effort clearly proves that recognizing speech and visuals may broaden access, helping those with physical limits engage more easily with machines. Findings show accurate motion handling and solid teamwork between camera tracking and vocal orders. The biggest issue mentioned is how it relies heavily on fixed lighting and backdrop settings to process images right. Still, other people on this field think using smart learning systems along with flexible visual tech might boost performance across different real-world situations.

2.6 Implementation of Pick and Place Robotic Arm Using Speech Processing and IoT

This study looks into combining speech with IoT in a robot arm built for picking and moving objects. Workers' demand for machines that do jobs perfectly or delays drives the idea forward. Instead of relying on buttons or touchscreens, this version runs on speech caught by Google Assistant using a Node-MCU linked to Wi-Fi. Commands travel from the cloud to motors through signal boosters, making action accordingly. Onboard infrared sensors spot items nearby so actions kick off only when object is present. Test outcomes confirm it's possible to run voice commands remotely using IoT systems, with decent speed plus correct execution. Still, findings show performance relies a lot on steady internet signals along with clear audio input — hinting that built-in standalone processing might boost quick reactions down the line.

2.7 Voice Controlled Object Grasping Robotic Arm for Visually Impaired Disabled Veterans

This study introduces a robot arm designed to help people with sight or mobility issues to do daily activities like picking things from refrigerator. It uses voice understanding and image recognition to work better. It turns speech into digital commands, spots items using YOLOv4, while a RealSense D455 sensor judges distance. Once the user speaks, the system links words to what's seen, then moves a DOBOT Magician arm — guided by personalized motion math — to complete the task. Tests show solid results when spotting and pulling objects in different settings. Though it cuts down on manual work, the study admits issues with tricky voice inputs and shifting light levels — pointing toward smarter language models and better camera tuning ahead.

2.8 Voice Controlled Camera-Assisted Pick and Place Robot Using Raspberry Pi

This study introduces a robotic setup that follows spoken cues while using visual tracking to handle pickup and sorting jobs without help. Running on a micro-controller equipped with a small camera, it spots items by traits such as hue or dimensions — then approaches, lifts, and places them. Control happens via smartphone interface offering two options: direct handling with taps or speech input, versus self-guided operation where the model identifies them. Its imaging method paired with decision logic achieved 95% correctness when responding to voice inputs, showing solid results indoors. The research shows combining visual analysis with spoken commands helps robots work more independently while cutting down on mistakes during everyday jobs — yet one downside is both the vision and speech tools can struggle when light levels shift or background sounds get too loud, making them less reliable outdoors or in busy spaces.

2.9 Smart Autonomous Wireless Gripper for Human-Robot Collaborative Operation

The objective of this research paper is to close a gap in the surgical field by developing an advanced automated gripper that automatically retrieves surgical instruments as directed through voice command. The methodology involved the use of axiomatic design where the required functionality is assigned to corresponding design parameters within the different subsystems that include Google cloud speech-to-text for voice command,

YOLOv5 algorithm based object detection implemented using an ESP-32 camera for identifying the surgical instrument, and ArUco marker to localize the coordinates of the surgical tool which are communicated wirelessly using TCP/IP socket connection to the Svaya SR-L16 robot. The testing phase demonstrated remarkable findings with 93% detection accuracy in regular operating theater lighting condition and more than 90% speech recognition accuracy in noisy environments, thus verifying the usability of the gripper in surgical environments. Nevertheless, the gripper assembly is made from ABS plastic material as opposed to the targeted medical grade PA12 due to printing equipment availability. Moreover, the performance of the developed system under varying ambient conditions requires further investigation.

2.10 Multimodal Human-Robot Interaction Using Gestures and Speech: A Case Study for Printed Circuit Board Manufacturing

This study presents a flexible, safe setup for human-robot teamwork in circuit board production. Instead of relying on one method, it combines motion sensing tools — the Leap Motion device works at about 95% precision for both hands — alongside the Tap Strap 2, which hits nearly 98% correctness thanks to an LSTM network. Voice commands run through Cellya's custom speech recognizer tuned for Spanish, cutting mistakes in half — from 44% down to 22% — while hitting close to 93% success rate. These inputs feed into a distributed control hub made with Node.js, sorting who can do what based on face scans processed by the LBPH technique. Access gets checked securely before allowing any action. Even as more people join, the structure holds up well, managing many users and devices together using MQTT messaging. Still, newcomers using the Tap Strap might see shaky results early because sensor drift affects gestures, so adjusting settings beforehand helps improve performance.

2.11 Robotic Arm for Pick and Place Application

The purpose of this paper was quite clear and realistic — creating an efficient, cost-effective, and programmable robotic arm having four degrees of freedom for carrying out basic pick and place operations, mainly for educational purposes. For designing the prototype, the authors have made use of CATIA V5.0 software for CAD design, constructed the framework using aluminum material which was light and robust, and used a microcontroller of NodeMCU type, powered by ESP8266 WiFi technology. The

latter received input from the Android application and produced PWM output controlling four Tower Pro MG945 servo motors at joints and one SG90 motor for gripping. The proposed device was implemented successfully and proven to be effective, cheap, and lightweight; moreover, the idea shows great promise for usage in educational laboratories, where it can be used for training students and for introducing them into the robotics and automation field. Nevertheless, it should be pointed out that the authors do not report about any quantitative characteristics, including performance tests results and load tests, which makes their conclusions questionable.

2.12 Voice-Controlled Pick and Place Robot

This research dives into how a voice-operated robot can grab and move items around. Its main goal? Letting people with physical disabilities handle tough factory jobs by speaking simple orders. Instead of wires, it uses an HC-05 Bluetooth chip to catch those spoken inputs. Commands get sorted out through an ATmega16 brain that runs the whole show. To spot what's in front of it, the bot leans on an ESP32-CAM for eyes. When tested, it worked smoothly — finding things, lifting them up, shifting positions, all without delays. The setup kicks off when someone uses an Android app to send signals — these travel over Bluetooth and get handled by a tiny onboard computer that tells the robot how to move or grab things. Tests showed it works almost every single time without hiccups. It nailed voice detection 95.7% right in noise-free rooms, grabbed items below 100 grams correctly 97.5% of attempts, plus reacted within just over a second on average. Designed mainly to help people with physical challenges gain more freedom at workspaces such as factories, storage areas, or office buildings.

Chapter 3

Methodology

The major work in our work was not simply adding separate AI tools, it was creating a unified setup so they can interact smoothly on the fly. The entire system is implemented in C++ and Python, running natively on our NVIDIA Jetson Orin Nano, with all core functions of audio capture, speech transcription, intent extraction, vision, and motion, which is built as an independent module. ESP32 micro-controller handles all low-level hardware interfacing by communicating with the Jetson Serially at 115200 baud rate connection. As it is set up this way, upgrading the vision module will not mean that we have to redo how voice or movement modules work. This section explains the exact tools we have picked and then walks through how the setup works across the three levels, perception, processing and actual movement.

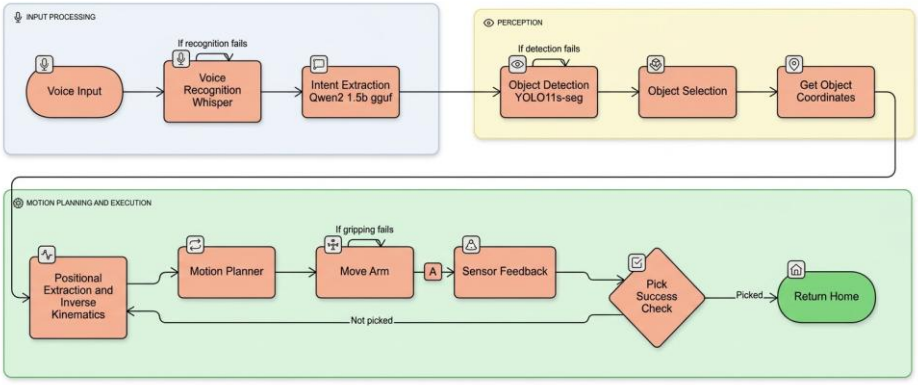

Figure 3.1: Architecture

3.1 The Software and Hardware Stack

The body of our robotarm is made from scratch using 3D printing, which gives us the ability to customize it as we require.

3.1.1 Hardware Configuration

Chassis: We have used 3D-printed pieces along with polycarbonate panels for the frame to keep it lightweight while staying strong.

Actuators: Four servo motors are used at base, shoulder, elbow, and the gripper. SG-90 and MG995 servo motors are used across the joints. All four servos are driven by an Adafruit PCA9685 PWM servo driver (I^2C address `0x40`) which operates at 50 Hz. Smooth motion of the arm is achieved by incrementally changing PWM value toward the target in steps of 10 every 30 ms to avoid the sudden jerks. The arm joints (base, shoulder, elbow) use a PWM range of 110–590. The gripper uses 125–400.

Sensors: A USB webcam fixed near the gripper provides the visual feed. A USB microphone connected via `plughw:2,0` captures audio. An **Adafruit VL53L1X** Time of Flight (TOF) sensor, connected via I^2C on the pins SDA = 4, SCL = 5, running at 115200 baud, is used for depth measurement. The TOF sensor operates in a demand-driven mode and returns a distance reading when it receives the character `'s'` over Serial.

Controller: We use a **NVIDIA Jetson Orin Nano** for all the major computations such as all AI inference, speech transcription, intent extraction, and vision entirely on-device without cloud dependency. We use a **ESP32** microcontroller which receives high-level commands from the Jetson over USB Serially and then translates them into PWM signals for controlling the servos and TOF sensor.

3.1.2 Software Stack

Simulation & Development — Custom IK/FK Solver, OpenCV

To safely develop and validate the robot's motion planning logic, a custom forward and inverse kinematics solver was implemented and tested in software before deploying to physical hardware.

Decision & Planning — Qwen2 1.5B (GGUF), Custom IK/FK Solver, OpenCV

Qwen2 1.5B (GGUF) extracts user intent from transcribed text entirely on-device. The custom IK/FK solver handles joint angle computation, path planning, and smooth motion execution. OpenCV is used for 2D to 3D coordinate transformations.

Perception & AI Core — whisper.cpp, YOLO11s-seg, OpenCV

whisper.cpp (`ggml-base.en-q5_0` model) runs on the Jetson Orin Nano's GPU to convert speech to text entirely in C++ without any cloud dependency. YOLO11s-seg performs real-time instance segmentation for accurate, high-speed general object detection and localisation.

Hardware Abstraction & Control — Arduino (C++), `arecord`, Adafruit PCA9685, VL53L1X libraries

All servo control and TOF sensing run on a dedicated Arduino. The Jetson communicates with the Arduino over USB Serial at 115200 baud, sending structured text commands.

Speech → `arecord` (WAV) → whisper.cpp (STT) → Qwen2 1.5B GGUF (Intent JSON) → YOLO11s-seg (Detection) → OpenCV + TOF (3D coordinates) → IK/FK Solver (Path Planning) → Arduino Serial (Servo + Gripper execution)

3.2 Layer 1: Perception

This layer is responsible for converting raw audio and video into structured, actionable data. We intentionally split this into two specialised sub-modules.

3.2.1 Audio and NLU

The audio pipeline is implemented entirely in C++ and split into three stages.

- **Recording (`rec till r.cpp`):** A master control loop (`master.cpp`) listens for keypresses in raw terminal mode. When the user presses `r`, it forks a child process that launches `arecord` targeting the USB microphone at `plughw:2,0`, recording in CD-quality WAV format to `recorded.wav`. Pressing `r` again sends `SIGINT` to stop the recording cleanly. A timestamped log (`record.log`) is written for every session.
- **Transcription (`whisper-call.cpp`):** The transcription module invokes whisper.cpp via `popen()`, running the quantised `ggml-base.en-q5_0` model directly on the Jetson Orin Nano's GPU. Output is filtered line-by-line to strip

whisper.cpp internal logs (`system_info`, `main:` prefixes), returning only clean transcribed text. A logging variant (`whisper-call-log.cpp`) appends timestamped transcripts to `transcript.log` for debugging.

- **Intent Extraction (Qwen2 1.5B GGUF):** The clean transcribed text is fed to Qwen2 1.5B (GGUF) with a structured prompt. The model acts solely as an intent parser, returning only a JSON object. For example, if the user says "grab the red cup", the output is *{*`"action": "GRAB", "object": "red cup"`*}*. This JSON is then passed directly to the vision module.

3.2.2 Vision

The vision pipeline uses YOLO11s-seg running continuously on the Jetson Orin Nano's GPU. Unlike standard bounding-box detectors, YOLO11s-seg performs instance segmentation, producing precise segmentation masks for each detected object. This allows more accurate centroid estimation, which directly improves the quality of the 3D coordinate calculation downstream.

YOLO11s-seg handles all detection tasks — including descriptive commands such as "grab the cup" — by matching the object label returned in the intent JSON to detected instances in the scene. The model operates at high frame rates on the Jetson's GPU, keeping the robot's scene awareness real-time and responsive.

3.3 Layer 2: Planning and Execution

Once the target object is localised in the 2D camera frame, the planning and execution layer takes over. This is a three-part process.

- **Coordinate Extraction (OpenCV + TOF):** The 2D pixel centroid from the segmentation mask is used to align the arm toward the object. The Jetson sends the character `'s'` to the Arduino over Serial; the Arduino queries the VL53L1X TOF sensor and returns the distance in millimetres. This depth value, combined with the 2D centroid, is used to compute the 3D world coordinates via trigonometric offsets in OpenCV.

- **Path Planning (Custom IK/FK Solver):** The 3D goal coordinate is passed to our custom Inverse Kinematics (IK) and Forward Kinematics (FK) solver. The IK solver computes the valid joint angles for the base, shoulder, and elbow given the target position. The FK solver verifies the resulting pose. Together with a smooth

path planning algorithm, the solver generates a continuous, jerk-free trajectory that moves the arm safely from its current position to the target.

- **Execution (Arduino Serial):** The Jetson sends structured Serial commands to the Arduino at 115200 baud. Arm joint angles are sent as `d <base> <shoulder> <elbow>` and gripper angle as `g <angle>`. The Arduino's `angleToPWM()` function maps each 0–180° angle to the correct PCA9685 PWM tick count. A non-blocking servo update loop running every 30 ms increments each servo's current PWM toward its target in steps of 10, producing smooth, jerk-free motion. The Arduino responds with `ok` on success or an error string if the command is malformed. Current PWM values for all four channels can be queried at any time using `?`.

Chapter 4

Results and Discussions

4.1 Objective

The goal of our testing was to make sure the feasibility and performance of each core module in our proposed architecture has been met. We focused our analysis on the "perception" components, the NLU and vision pipelines, as their real-world performance is the primary bottleneck for a system of this nature. This chapter presents the results of these tests and discusses the critical implementation decisions made in response to our findings.

4.2 NLU Pipeline: whisper.cpp and Qwen2 1.5B (GGUF)

Our first motive was to make sure that the complete speech to text and intent extraction pipeline was running entirely on the Jetson Orin Nano in C++. The goal was to make sure that the setup is able to record audio, transcribe it, and return a clear JSON output.

The speech-to-text models assessed include Vosk, Google Cloud Speech-to-Text, and whisper.cpp. Whisper was superior to other models in terms of accuracy and tolerance to noise. The quantised ggml-base.en-q5 0 model was used due to its reasonable trade-off between accuracy and speed using the Jetson Orin Nano hardware. The transcript is generated using the transcription module (whisper-call.cpp), which calls the whisper.cpp model using popen() and removes logging information.

The rec_till_r.cpp program records the sound from the USB microphone using arecord with CD-quality sound (in WAV format). The master.cpp program controls the user interface process: pressing r will start and stop the recording, while pressing p will play

the recording and pressing q will exit the program. A logging version (whisper-call-log.cpp) appends timestamped logs to transcript.log for debugging purposes.

The transcribed text is then passed to Qwen2 1.5B (GGUF), which acts purely as an intent parser running entirely on-device on the Jetson Orin Nano. The model successfully captures full descriptive phrases rather than just keywords essential for the visual grounding task that follows. We achieved **90% accuracy** for structured JSON intent extraction.

```
Extracting intents: 100%|##########| 100/100 [02:22<00:00,  1.42s/it]

=== Overall Intent Accuracy ===
99.00%

=== Action Classification Report ===
              precision    recall  f1-score   support

        bring       1.00      1.00      1.00        20
         find       1.00      1.00      1.00        20
          get       1.00      1.00      1.00        20
       locate       1.00      1.00      1.00        20
         pick       1.00      1.00      1.00        20

     accuracy                           1.00       100
    macro avg       1.00      1.00      1.00       100
 weighted avg       1.00      1.00      1.00       100


=== Object Classification Report ===
              precision    recall  f1-score   support

         ball       1.00      1.00      1.00        12
         book       1.00      1.00      1.00         9
       bottle       1.00      1.00      1.00         9
          box       1.00      1.00      1.00         6
        chair       1.00      1.00      1.00        12
          cup       1.00      1.00      1.00         7
          key       1.00      1.00      1.00         8
          pen       1.00      1.00      1.00        11
        phone       1.00      1.00      1.00        10
          toy       1.00      1.00      1.00        16

     accuracy                           1.00       100
    macro avg       1.00      1.00      1.00       100
 weighted avg       1.00      1.00      1.00       100
```

Figure 4.1: Qwen2 1.5B (GGUF) Test Score

4.3 Vision Pipeline: YOLO11s-seg

Our vision testing was designed to confirm YOLO11s-seg's viability as the sole detection and localisation model for the system.

4.3.1 YOLO11s-seg: Instance Segmentation for Object Localisation

We tested YOLO11s-seg on the Jetson Orin Nano's GPU across a variety of scenes containing multiple objects. The model performed exceptionally well, achieving stable high-frame-rate inference well within our real-time requirements. Unlike standard bounding-box detectors, YOLO11s-seg produced precise segmentation masks for each

```
PS C:\Users\Naras\Desktop\Whisper\Speech-and-Vision-Integrated-Robotic-Arm-mai
/Python/Python313/python.exe c:/Users/Naras/Desktop/Whisper/Speech-and-Vision-
 Model 'medium.en' loaded.
Press [spacebar] to toggle recording ON/OFF. Ctrl+C to exit.

 Recording...
Recording stopped. Transcribing...
search for a green apple


 Recording...
Recording stopped. Transcribing...
Grab a red ball.


 Recording...
Recording stopped. Transcribing...
Pick the blue pen.


 Recording...
Recording stopped. Transcribing...
get the pen on the right.
```

Figure 4.2: Whisper Speech-to-Text Translation

```
[ Prompt: 207.2 t/s | Generation: 29.8 t/s ]

> "pick a blue pen"

```json
{
  "action": "pick",
  "object": "blue pen"
}
```

[Prompt: 203.8 t/s | Generation: 30.2 t/s]

> "pick a deformed ball"

```json
{
  "action": "pick",
  "object": "deformed ball"
}
```

[Prompt: 212.1 t/s | Generation: 29.9 t/s]

>
```

Figure 4.3: Qwen2 1.5B (GGUF) JSON Outputs for sample inputs

detected object, enabling more accurate centroid extraction for downstream 3D coordinate calculation.

The model successfully identified and segmented all objects in the test scene simultaneously. By matching the object label from the Qwen2 intent JSON to detected instances, the system could reliably isolate the correct target even in cluttered scenes.

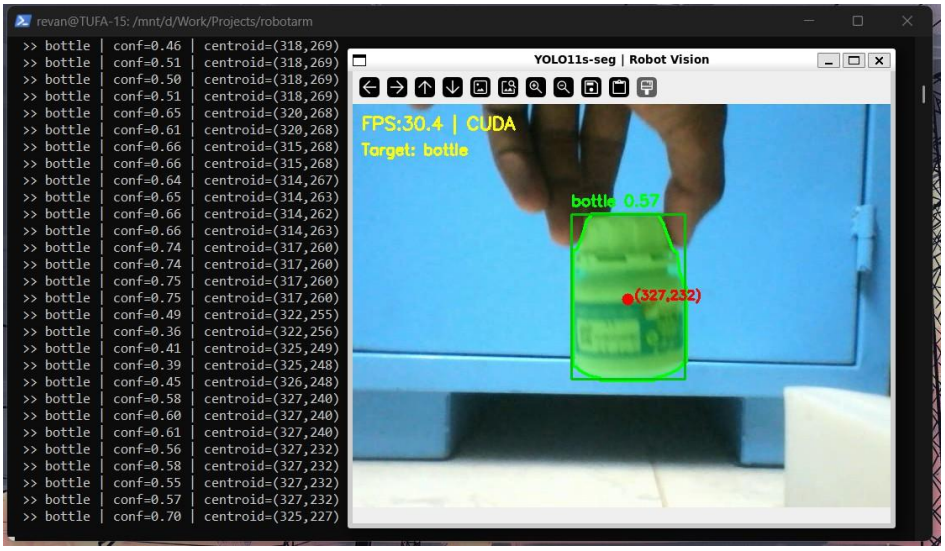


Figure 4.4: YOLO11s-seg performing instance segmentation on the scene

## 4.4 Hardware Setup

The physical prototype consists of a 3D-printed robotic arm mounted on a poly-carbon base platform elevated by four cylindrical standoffs. The arm has three degrees of freedom — base rotation, shoulder, and elbow — all driven by SG-90 and MG995 servo motors. The PCA9685 16-channel PWM servo driver board and the Arduino microcontroller are mounted directly on the base platform, with all servo wiring routed neatly along the arm structure using multicolour jumper cables. The VL53L1X TOF sensor is mounted on the gripper, facing forward toward the workspace for accurate depth measurement. A LiPo battery pack is also mounted on the base platform to power the servo subsystem independently. The NVIDIA Jetson Orin Nano — with its active cooling fan — is placed externally beside the base and connected to the Arduino via a USB cable, handling all AI inference on-device.

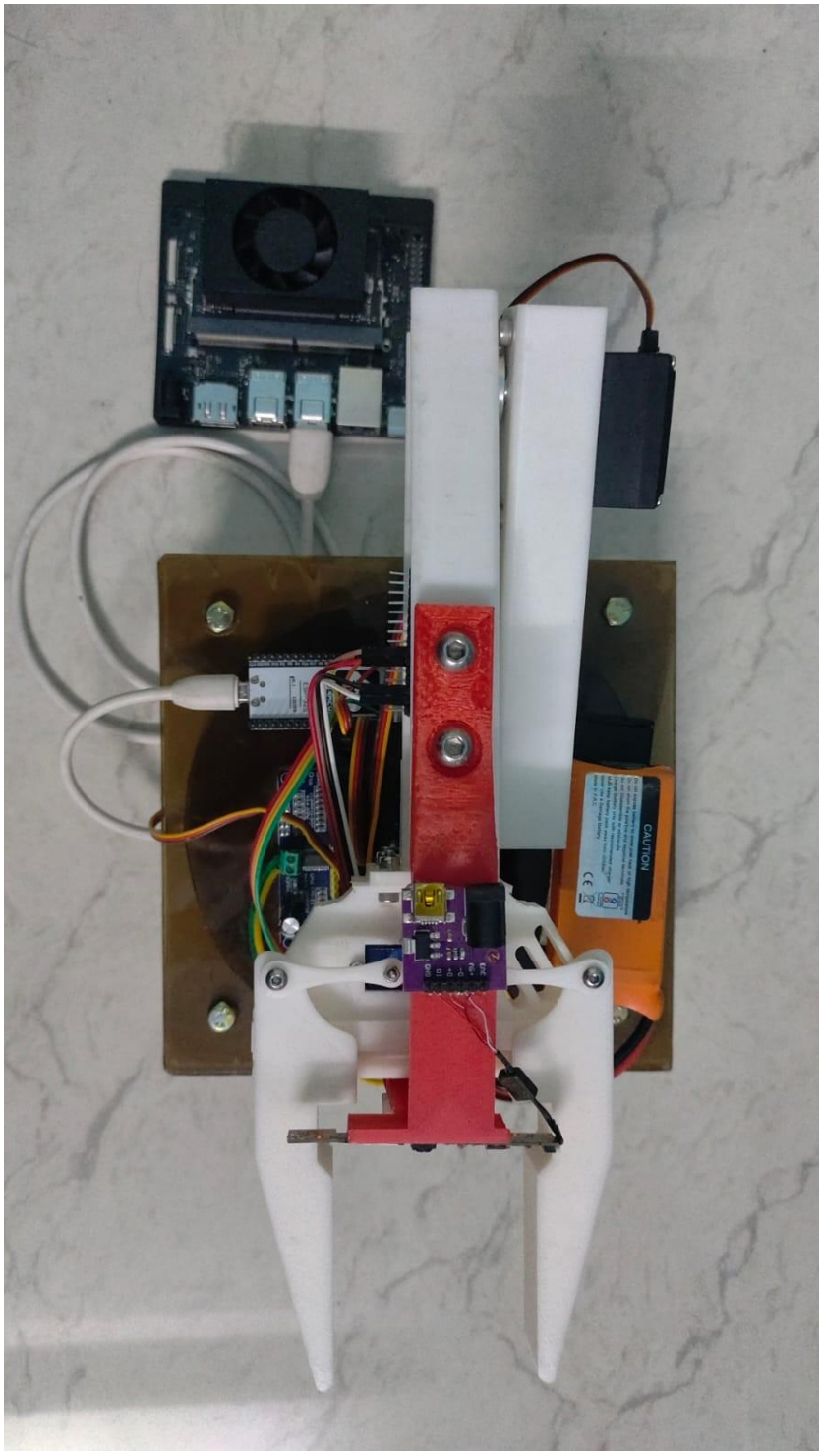


Figure 4.5: Top View of the Hardware Setup

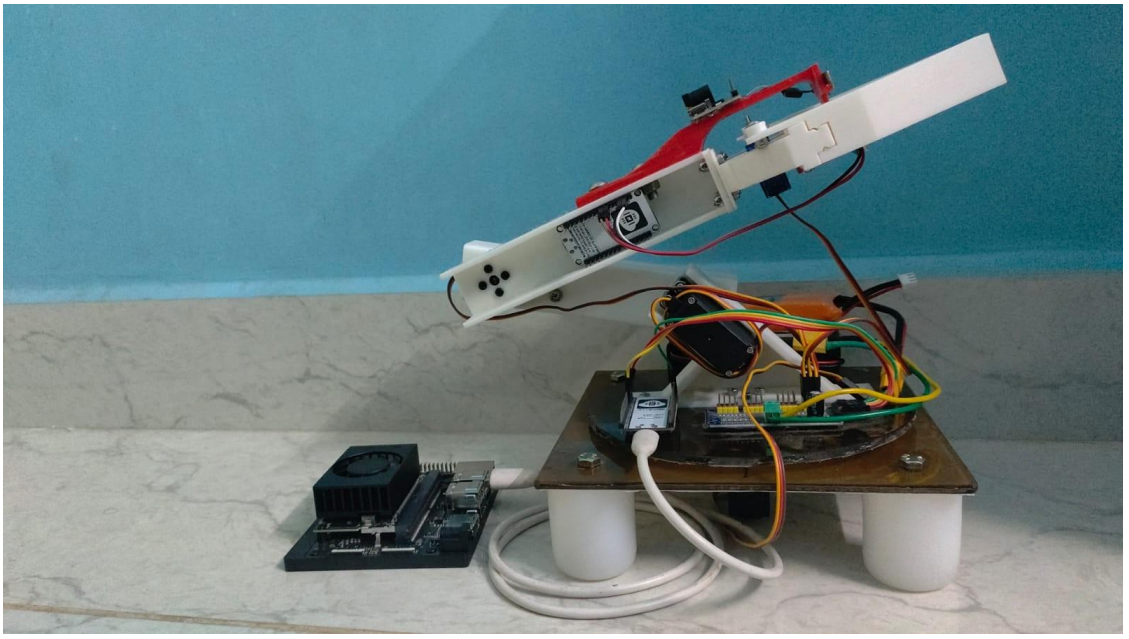

Figure 4.6: Side View of the Hardware Setup

# Chapter 5

# Conclusion and Future Scope

This project successfully demonstrates a speech- and vision-controlled robotic arm capable of interpreting natural commands and executing physical actions in real time, entirely on-device without cloud dependency. Whisper.cpp was chosen for speech recognition after evaluating Vosk and Google Cloud STT, as it provided higher accuracy and stable transcription on the NVIDIA Jetson Orin Nano. Qwen2 1.5B (GGUF) was selected as the intent extraction model for its ability to parse full natural language descriptions into structured JSON outputs efficiently on edge hardware. YOLO11s-seg was chosen as the vision model for its dual capability of high-speed detection and precise instance segmentation, enabling accurate object centroid extraction for downstream coordinate computation.

The system was implemented on a budget-friendly pipeline, using the Jetson Orin Nano for all on-device AI inference, a USB webcam and VL53L1X TOF sensor for spatial understanding, and a dedicated Arduino microcontroller for hardware abstraction — receiving structured Serial commands from the Jetson and converting them to PWM signals for the PCA9685-driven servo motors. Motion execution was achieved using a custom IK/FK solver and smooth path planning algorithms, enabling the robotic arm to identify objects, understand voice instructions, and perform smooth coordinated movements. Overall, the project proves that an affordable, real-time multimodal human-robot interaction system is feasible with careful model and hardware selection.

The system can be improved further by integrating advanced natural language understanding models so the robotic arm can interpret more complex and conversational voice commands. A custom-trained object-attribute detection model can be developed to increase accuracy for specific items and fine-grained features that general pretrained models may miss. The entire pipeline can also be upgraded to an end-to-end multimodal framework that directly connects speech, vision, and action, reducing latency

and improving robustness. Depth perception and localisation can be enhanced with better calibration, multi-camera setups, or SLAM-based mapping. Hardware optimisation could enable deployment on lower-cost boards with reduced power consumption, making the system more scalable and affordable. In addition, adding dynamic obstacle avoidance, improved path-planning, and safety features such as tactile or force sensors would make the arm more reliable for real-world human-robot interaction. Continuous learning techniques could also be introduced, allowing the system to learn new objects and tasks over time, increasing adaptability and long-term usability.

# Chapter 6

# Appendix

## 6.1 master.cpp

```cpp
#include <iostream>
#include <unistd.h>
#include <termios.h>
#include <sys/wait.h>

static struct termios oldt;

void restore_terminal() {
    tcsetattr(STDIN_FILENO, TCSANOW, &oldt);
}

void setup_terminal() {
    struct termios newt;
    tcgetattr(STDIN_FILENO, &oldt);
    newt = oldt;
    newt.c_lflag &= ~(ICANON | ECHO);
    tcsetattr(STDIN_FILENO, TCSANOW, &newt);
    atexit(restore_terminal);
}

void run_program(const char* prog) {
    pid_t pid = fork();
    if (pid == 0) {
        execl(prog, prog, (char*)nullptr);
        _exit(1);   // exec failed
    }
    waitpid(pid, nullptr, 0);
}

```

Figure 6.1: master.cpp – Part 1

```cpp

int main() {
    setup_terminal();

    std::cout << "Menu:\n";
    std::cout << "  r → Record an audio\n";
    std::cout << "  p → Play the recorded audio\n";
    std::cout << "  q → quit\n";

    while (true) {
        char c;
        if (read(STDIN_FILENO, &c, 1) <= 0)
            continue;

        std:: cout << "\n";

    if (c == 'r') {
            run_program("./rec_tr");
        }
        else if (c == 'p') {
            run_program("./play");
        }
        else if (c == 'q') {
            std::cout << "Exiting... Bye 👋\n";
            break;
        }
    }

    return 0;
}

```

Figure 6.2: master.cpp – Part 2

6.2 play_rec.cpp

```cpp
#include <iostream>
#include <unistd.h>
#include <sys/wait.h>

int main() {

    pid_t pid = fork();
    if (pid == 0) {
        execlp("aplay", "aplay",
               "-D", "plughw:2,0",
               "recorded.wav",
               (char*)nullptr);
        _exit(1);
    }

    waitpid(pid, nullptr, 0);

    std::cout << "✅ Playback finished\n";
    return 0;
}

```

Figure 6.3: play_rec.cpp

6.3 rec_till_r.cpp

```
#include <iostream>
#include <unistd.h>
#include <termios.h>
#include <sys/wait.h>
#include <signal.h>
#include <fcntl.h>
#include <ctime>
#include <cstring>
static struct termios oldt;

void restore_terminal() {
    tcsetattr(STDIN_FILENO, TCSANOW, &oldt);
}

void setup_terminal() {
    struct termios newt;
    tcgetattr(STDIN_FILENO, &oldt);
    newt = oldt;
    newt.c_lflag &= ~(ICANON | ECHO);
    tcsetattr(STDIN_FILENO, TCSANOW, &newt);
    atexit(restore_terminal);
}

void write_timestamp(int fd) {
    std::time_t now = std::time(nullptr);
    char buf[128];
    std::strftime(buf, sizeof(buf),
                  "\n===== %Y-%m-%d %H:%M:%S =====\n",
                  std::localtime(&now));
    write(fd, buf, strlen(buf));
}

```

Figure 6.4: rec_till_r.cpp – Part 1

```cpp
int main() {
    setup_terminal();

    pid_t pid = fork();
    if (pid == 0) {
        int log_fd = open("record.log",
                          O_WRONLY | O_CREAT | O_APPEND,
                          0644);
        if (log_fd >= 0) {
            write_timestamp(log_fd);
            dup2(log_fd, STDOUT_FILENO);
            dup2(log_fd, STDERR_FILENO);
            close(log_fd);
        }

        execlp("arecord", "arecord",
               "-D", "plughw:2,0",
               "-f", "cd",
               "-t", "wav",
               "recorded.wav",
               (char*)nullptr);
        _exit(1);
    }

    std::cout << "🎙 Recording... press 'r' again to stop\n";
    char c;
    while (read(STDIN_FILENO, &c, 1) > 0) {
        if (c == 'r') break;
    }

    kill(pid, SIGINT);
    waitpid(pid, nullptr, 0);

    std::cout << "✅ Saved recorded.wav\n";
    return 0;
}
```

Figure 6.5: rec_till_r.cpp – Part 2

6.4 whisp_embed_nu.cpp

```cpp
#include <iostream>
#include <unistd.h>
#include <termios.h>
#include <sys/wait.h>

static struct termios oldt;

void restore_terminal() {
    tcsetattr(STDIN_FILENO, TCSANOW, &oldt);
}

void setup_terminal() {
    struct termios newt;
    tcgetattr(STDIN_FILENO, &oldt);
    newt = oldt;
    newt.c_lflag &= ~(ICANON | ECHO);
    tcsetattr(STDIN_FILENO, TCSANOW, &newt);
    atexit(restore_terminal);
}

void run_program(const char* prog) {
    pid_t pid = fork();
    if (pid == 0) {
        execl(prog, prog, (char*)nullptr);
        _exit(1);   // exec failed
    }
    waitpid(pid, nullptr, 0);
}

```

Figure 6.6: whisp_embed_nu.cpp – Part 1

```
int main() {
    setup_terminal();

    std::cout << "Menu:\n";
    std::cout << "  r → Record an audio\n";
    std::cout << "  t → Transcribe the audio\n";
    std::cout << "  q → quit\n";

    while (true) {
        char c;
        if (read(STDIN_FILENO, &c, 1) <= 0)
            continue;

        std:: cout << "\n";

    if (c == 'r') {
            run_program("./rec_tr");
        }
        else if (c == 't') {
            run_program("./transcribe");
        }
        else if (c == 'q') {
            std::cout << "Exiting... Bye 👋\n";
            break;
        }
    }

    return 0;
}
```

Figure 6.7: whisp_embed_nu.cpp – Part 2

6.5 whisper_call.cpp

```cpp
#include <iostream>
#include <cstdio>
#include <memory>
#include <array>
#include <string>

std::string run_whisper() {
    std::string command =
        "~/Arm/whisper.cpp/build/bin/whisper-cli "
        "-m ~/Arm/whisper.cpp/models/ggml-base.en-q5_0.bin "
        "-f recorded.wav "
        "-nt -np";

    std::array<char, 256> buffer;
    std::string result;

    FILE* pipe = popen(command.c_str(), "r");
    if (!pipe) {
        return "ERROR: Failed to run whisper";
    }

    while (fgets(buffer.data(), buffer.size(), pipe) != nullptr) {
        std::string line(buffer.data());

        // Skip unwanted lines (logs, system info)
        if (line.find("whisper") != std::string::npos) continue;
        if (line.find("system_info") != std::string::npos) continue;
        if (line.find("main:") != std::string::npos) continue;

        result += line;
    }

    pclose(pipe);
    return result;
}

int main() {
    std::cout << "🔴 Transcribing...\n";

    std::string text = run_whisper();

    std::cout << "✅ Output:\n" << text << std::endl;

    return 0;
}
```

Figure 6.8: whisper_call.cpp

6.6 whisper_call_log.cpp

```cpp
#include <iostream>
#include <cstdio>
#include <memory>
#include <array>
#include <string>
#include <fcntl.h>
#include <unistd.h>
#include <ctime>
#include <cstring>

// ◆ Write timestamp (same as your recorder)
void write_timestamp(int fd) {
    std::time_t now = std::time(nullptr);
    char buf[128];
    std::strftime(buf, sizeof(buf),
                  "\n===== %Y-%m-%d %H:%M:%S =====\n",
                  std::localtime(&now));
    write(fd, buf, strlen(buf));
}

std::string run_whisper() {
    std::string command =
        "/home/atomic/Arm/whisper.cpp/build/bin/whisper-cli "
        "-m /home/atomic/Arm/whisper.cpp/models/ggml-base.en-q5_0.bin "
        "-f recorded.wav "
        "-nt -np";

    std::array<char, 256> buffer;
    std::string result;

    // ◆ Open log file
    int log_fd = open("transcript.log",
                      O_WRONLY | O_CREAT | O_APPEND,
                      0644);

    if (log_fd >= 0) {
        write_timestamp(log_fd);
    }

```

Figure 6.9: whisper_call_log.cpp – Part 1

```
40      FILE* pipe = popen(command.c_str(), "r");
41      if (!pipe) {
42          if (log_fd >= 0) {
43              const char* err = "ERROR: Failed to run whisper\n";
44              write(log_fd, err, strlen(err));
45              close(log_fd);
46          }
47          return "ERROR: Failed to run whisper";
48      }
49
50      while (fgets(buffer.data(), buffer.size(), pipe) != nullptr) {
51          std::string line(buffer.data());
52
53          // 🔹 Log EVERYTHING
54          if (log_fd >= 0) {
55              write(log_fd, line.c_str(), line.size());
56          }
57
58          // 🔹 Filter only clean text for return
59          if (line.find("whisper") != std::string::npos) continue;
60          if (line.find("system_info") != std::string::npos) continue;
61          if (line.find("main:") != std::string::npos) continue;
62
63          result += line;
64      }
65
66      pclose(pipe);
67
68      if (log_fd >= 0) {
69          close(log_fd);
70      }
71
72      return result;
73  }
74
75  int main() {
76      std::cout << "🧠 Transcribing...\n";
77
78      std::string text = run_whisper();
79
80      std::cout << "✅ Output:\n" << text << std::endl;
81
82      return 0;
83  }
```

Figure 6.10: whisper_call_log.cpp – Part 2

6.7 Intent Client Server Code

```cpp
#include <curl/curl.h>
#include <iostream>
#include <string>
#include <nlohmann/json.hpp>

using json = nlohmann::json;

// ◆ callback
size_t WriteCallback(void* contents, size_t size, size_t nmemb, std::string* output) {
    output->append((char*)contents, size * nmemb);
    return size * nmemb;
}

json get_intent(const std::string& input) {
    CURL* curl = curl_easy_init();
    std::string response;

    std::string body =
        "{"
        "\"messages\":["
        "{\"role\":\"system\",\"content\":\"Extract intent as JSON with action and object only.\"},"
        "{\"role\":\"user\",\"content\":\"" + input + "\"}"
        "],"
        "\"json_schema\":{"
        "\"type\":\"object\","
        "\"properties\":{"
        "\"action\":{\"type\":\"string\"},"
        "\"object\":{\"type\":\"string\"}"
        "},"
        "\"required\":[\"action\",\"object\"]"
        "}"
        "}";

    struct curl_slist* headers = NULL;
    headers = curl_slist_append(headers, "Content-Type: application/json");

```

Figure 6.11: Intent Client Server Code – Part 1

```
36
37      curl_easy_setopt(curl, CURLOPT_URL, "http://localhost:8080/v1/chat/completions");
38      curl_easy_setopt(curl, CURLOPT_HTTPHEADER, headers);
39      curl_easy_setopt(curl, CURLOPT_POSTFIELDS, body.c_str());
40      curl_easy_setopt(curl, CURLOPT_WRITEFUNCTION, WriteCallback);
41      curl_easy_setopt(curl, CURLOPT_WRITEDATA, &response);
42
43      curl_easy_perform(curl);
44
45      curl_easy_cleanup(curl);
46      curl_slist_free_all(headers);
47
48      // 🔥 Step 1: parse outer JSON
49      json outer = json::parse(response);
50
51      // 🔥 Step 2: extract inner string
52      std::string content = outer["choices"][0]["message"]["content"];
53
54      // 🔥 Step 3: parse inner JSON
55      json final_json = json::parse(content);
56
57      return final_json;
58  }
59
60  int main() {
61      json result = get_intent("pick the blue pen above the phone");
62
63      std::cout << result << std::endl;
64
65      return 0;
66  }
67
```

Figure 6.12: Intent Client Server Code – Part 2

6.8 Qwen + Whisper Pipeline Code

```
#include <iostream>
#include <unistd.h>
#include <termios.h>
#include <sys/wait.h>
#include <string>

#include "transcribe.hpp"

static struct termios oldt;

// ◆ terminal
void restore_terminal() {
    tcsetattr(STDIN_FILENO, TCSANOW, &oldt);
}

void setup_terminal() {
    struct termios newt;
    tcgetattr(STDIN_FILENO, &oldt);
    newt = oldt;
    newt.c_lflag &= ~(ICANON | ECHO);
    tcsetattr(STDIN_FILENO, TCSANOW, &newt);
    atexit(restore_terminal);
}

// ◆ run program (recorder)
void run_program(const char* prog) {
    pid_t pid = fork();
    if (pid == 0) {
        execl(prog, prog, (char*)nullptr);
        _exit(1);
    }
    waitpid(pid, nullptr, 0);
}

// ◆ run intent client
void run_client(const std::string& text) {
    pid_t pid = fork();

    if (pid == 0) {
        execl("./client", "./client", text.c_str(), (char*)nullptr);
        _exit(1);
    }

    waitpid(pid, nullptr, 0);
}
```

Figure 6.13: Qwen + Whisper Pipeline Code – Part 1

```
47  int main() {
48      setup_terminal();
49      std::string last_text = "";
50      std::cout << "Menu:\n";
51      std::cout << "  r → Record audio\n";
52      std::cout << "  t → Transcribe\n";
53      std::cout << "  i → Intent\n";
54      std::cout << "  q → Quit\n";
55
56      while (true) {
57          char c;
58          if (read(STDIN_FILENO, &c, 1) <= 0)
59              continue;
60
61          std::cout << "\n";
62
63          if (c == 'r') {
64              run_program("./rec_tr");
65          }
66          else if (c == 't') {
67              std::cout << "🧠 Transcribing...\n";
68
69              last_text = run_whisper();   // 🔥 DIRECT CALL
70
71              std::cout << "✅ Text:\n" << last_text << "\n";
72          }
73          else if (c == 'i') {
74              if (last_text.empty()) {
75                  std::cout << "⚠️ No transcription available\n";
76                  continue;
77              }
78
79              std::cout << "🚀 Running intent...\n";
80              run_client(last_text);
81          }
82          else if (c == 'q') {
83              std::cout << "Bye 👋\n";
84              break;
85          }
86      }
87      return 0;
88  }
89
```

Figure 6.14: Qwen + Whisper Pipeline Code – Part 2

6.9 FK / IK Solver Code

```python
"""
FK and IK for your 3-DOF planar arm + base yaw.
Angles are in radians, lengths in mm.
"""

from typing import Tuple, List, Dict
import numpy as np
import math

# ---------- GLOBAL CONSTANTS ----------
D1 = 50.0     # base height
L2 = 110.0    # shoulder link
L3 = 260.0    # elbow link
Z_MIN = -50.0 # forbidden end-effector z region (z < -50)

# Default joint limits (degrees)
JOINT_LIMITS = {
    "theta1": (0, 180),  # 0 => 0 => theta1
    "theta2": (-180, 0), # 0 => 180 => 180 + theta2
    "theta3": (-45, 135) # 0 => 135 => 135 - theta3
}

def Rz(a):
    """Rotation about Z (radians)"""
    ca = np.cos(a); sa = np.sin(a)
    return np.array([[ca, -sa, 0.0],
                     [sa,  ca, 0.0],
                     [0.0, 0.0, 1.0]])

def Ry(a):
    """Rotation about Y with MATLAB convention used in your code:
       [ cos  0  sin
         0    1   0
        -sin  0  cos ]
    """
    ca = np.cos(a); sa = np.sin(a)
    return np.array([[ ca, 0.0,  sa],
                     [0.0, 1.0, 0.0],
                     [-sa, 0.0,  ca]])

```

Figure 6.15: FK/IK Code – Part 1

```python

def homogeneous(R, t):
    """Build a 4x4 homogeneous transform from 3x3 R and 3x1 t"""
    T = np.eye(4)
    T[0:3, 0:3] = R
    T[0:3, 3] = np.asarray(t).reshape(3,)
    return T

def fk(thetas_deg):
    """

    Inputs:
        D1, L2, L3 : link lengths (same units as thetas' position units)
        thetas_deg : iterable/array [t1_deg, t2_deg, t3_deg]

    Returns:
        O_all : numpy array shape (4,5) of homogeneous origins (columns)
                columns: base, after Rz, after Transz(D1), after L2, EE (full)
        R_all : list of 4 rotation matrices (3x3) for frames:
                [base (I), after Rz, after Transz(D1), after L2]
    """
    th = np.deg2rad(thetas_deg)
    t1, t2, t3 = th[0], th[1], th[2]

    # transforms
    T1 = homogeneous(Rz(t1), [0.0, 0.0, 0.0])              # base yaw
    T2 = T1 @ homogeneous(np.eye(3), [0.0, 0.0, D1])       # translate up D1
    T3 = T2 @ homogeneous(Ry(t2), [0.0, 0.0, 0.0])         # shoulder pitch
    T4 = T3 @ homogeneous(np.eye(3), [L2, 0.0, 0.0])       # translate along x by L2
    T5 = T4 @ homogeneous(Ry(t3), [0.0, 0.0, 0.0])         # elbow pitch
    T6 = T5 @ homogeneous(np.eye(3), [L3, 0.0, 0.0])       # translate along x by L3 (tool)
    Tfull = T6

    O_all = Tfull @ np.array([0.0, 0.0, 0.0, 1.0])

    return O_all[:3]


```

Figure 6.16: FK/IK Code – Part 2

```python

# ------------------- INVERSE KINEMATICS (WITH LIMITS) -------------------
def ik(x: float, y: float, z: float,
       limits: dict = JOINT_LIMITS,
       eps: float = 1e-9) -> List[Dict]:
    """
    Analytic IK with joint limit and z-constraint filtering.

    Args:
        x, y, z: target coordinates (mm)
        limits: dict of joint limit tuples in degrees
        eps: numerical clamp tolerance

    Returns:
        List of valid IK solution dicts:
          {
            'theta': np.array([t1, t2, t3]),
            'elbow': 'up' or 'down'
          }
        Empty list if no valid solutions.
    """
    solutions = []

    # ---- 1) Base angle ----
    theta1 = math.atan2(y, x)
    r = math.hypot(x, y)
    pz = z - D1

    # Convert to standard 2-link IK frame
    tx = r
    tz = -pz

    # distance from shoulder
    D = math.hypot(tx, tz)

    # ---- Reachability check ----
    if D > L2 + L3 + eps or D < abs(L2 - L3) - eps:
        return []  # nothing reachable

    # ---- 2) Solve elbow angle theta3 ----
    cos_t3 = (tx * tx + tz * tz - L2 * L2 - L3 * L3) / (2 * L2 * L3)
    cos_t3 = max(-1.0, min(1.0, cos_t3))  # clamp
    sin_t3_pos = math.sqrt(max(0.0, 1.0 - cos_t3 * cos_t3))
    sin_t3_neg = -sin_t3_pos

```

Figure 6.17: FK/IK Code – Part 3

```python

    t3_list = [
        math.atan2(sin_t3_pos, cos_t3),  # elbow-up
        math.atan2(sin_t3_neg, cos_t3)   # elbow-down
    ]

    # ---- Try both elbow configurations ----
    for t3 in t3_list:
        # Shoulder angle
        gamma = math.atan2(L3 * math.sin(t3), L2 + L3 * math.cos(t3))
        phi = math.atan2(tz, tx)
        t2 = phi - gamma

        # Build candidate
        cand = np.array([theta1, t2, t3], dtype=float)

        # ---- Joint limit filtering ----
        t_deg = np.degrees(cand)
        lim_ok = (
            limits["theta1"][0] <= t_deg[0] <= limits["theta1"][1] and
            limits["theta2"][0] <= t_deg[1] <= limits["theta2"][1] and
            limits["theta3"][0] <= t_deg[2] <= limits["theta3"][1]
        )

        if not lim_ok:
            continue

        # ---- z constraint filtering ----
        x_fk, y_fk, z_fk = fk(cand)
        if z_fk < Z_MIN:
            continue

        solutions.append({
            "theta": cand,
            "elbow": "up" if t3 >= 0 else "down"
        })

    return solutions

```

Figure 6.18: FK/IK Code – Part 4

```python


if __name__ == "__main__":

    # A concrete simple test:
    thetas_deg = [53.99998084, -25.35131794,  18.00581928]
    O_all = fk(thetas_deg)
    np.set_printoptions(precision=6, suppress=True)
    print(f"deg : {thetas_deg} \nfk : {O_all}\n\n")

    # A concrete simple test:
    thetas_deg = [0, 0,  0]
    O_all = fk(thetas_deg)
    np.set_printoptions(precision=6, suppress=True)
    print(f"deg : {thetas_deg} \nfk : {O_all}\n\n")

    target = (210.0, 289.04, 130.34)
    sols = ik(*target)
    print(f"Target {target}, Solutions found: {len(sols)}")
    for s in sols:
        t = s['theta']
        print(f" elbow={s['elbow']}  angles(deg)={np.degrees(t)}  -> FK={fk(np.degrees(t))}")


```

Figure 6.19: FK/IK Code – Part 5

6.10 Test Angle Control Code

```python
import serial
import time
from fk_ik_mat import fk, ik
import numpy as np

PORT = "COM7"
BAUD = 115200              # Must match the Arduino Serial.begin(115200)
DELAY_BETWEEN_POINTS = 3   # seconds between each command

ser = serial.Serial(PORT, BAUD, timeout=1)
time.sleep(2)  # allow board reset
print(f"Connected to {PORT} at {BAUD} baud.\n")

def send_point(theta0, theta1, theta2, ser=ser):
    """Send one point to Arduino in format [a, b, c]\\n"""
    msg = f"[{theta0}, {theta1+180}, {135-theta2}]\n"
    ser.write(msg.encode())
    print(f"Sent: {msg.strip()}")

    # Optionally wait for Arduino response
    time.sleep(0.1)
    while ser.in_waiting:
        line = ser.readline().decode(errors='ignore').strip()
        if line:
            print("Arduino:", line)

    # time.sleep(DELAY_BETWEEN_POINTS)
```

Figure 6.20: Test Angle Control Code – Part 1

```python

def call_route(target):
    sols = ik(*target)
    print(f"Target {target}, Solutions found: {len(sols)}")

    for s in sols:
        t = np.degrees(s['theta'])
        if (s['elbow'] == 'up'):
            print(f" angles(deg)={t}  -> FK={t}")
            send_point(*t)
            return t

if __name__ == "__main__":

    while True:
        print("Axes,.", end='')
        inp = input()
        t = tuple(map(float, inp.split()))
        try:
            call_route(t)
        except KeyboardInterrupt:
            print("closed")
```

Figure 6.21: Test Angle Control Code – Part 2

6.11 YOLO Object Detection Code

```python
import cv2, sys, time, numpy as np, torch
from ultralytics import YOLO
import os

TARGET      = sys.argv[1] if len(sys.argv) > 1 else None
SOURCE      = sys.argv[2] if len(sys.argv) > 2 else "/tmp/test_objects.mp4"
CONF        = 0.25
SAVE_OUTPUT = True
OUTPUT_PATH = "output_detected.mp4"

# ── Auto device + model selection ──────────────────────────────────────
device = "cuda" if torch.cuda.is_available() else "cpu"

# Use TensorRT engine on Jetson if available, fallback to .pt
ENGINE_PATH = "models/yolo/yolo11s-seg.engine"
PT_PATH     = "models/yolo/yolo11s-seg.pt"
MODEL_PATH  = ENGINE_PATH if os.path.exists(ENGINE_PATH) else PT_PATH

print(f"[YOLO] Device : {device}")
print(f"[YOLO] Model  : {MODEL_PATH}")
print(f"[YOLO] Source : {SOURCE}")
print(f"[YOLO] Target : '{TARGET or 'all objects'}'")

# ── IMPORTANT: disable CUDA sync to avoid WSL2 crash ──────────────────
if device == "cuda":
    import os
    os.environ["CUDA_LAUNCH_BLOCKING"] = "0"

model = YOLO(MODEL_PATH)
model.to(device)

cap = cv2.VideoCapture(SOURCE)
if not cap.isOpened():
    print(f"[ERROR] Cannot open: {SOURCE}")
    sys.exit(1)

fw     = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
fh     = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps_in = cap.get(cv2.CAP_PROP_FPS) or 25.0
writer = None
if SAVE_OUTPUT:
    fourcc = cv2.VideoWriter_fourcc(*"mp4v")
    writer = cv2.VideoWriter(OUTPUT_PATH, fourcc, fps_in, (fw, fh))
    print(f"[YOLO] Saving → {OUTPUT_PATH}")

```

Figure 6.22: YOLO Object Detection Code – Part 1

```python
class ObjectTracker:
    def _init_(self, max_lost=8):
        self.last_cx = self.last_cy = self.last_pts = self.last_conf = None
        self.lost_frames = 0
        self.max_lost = max_lost

    def update(self, cx, cy, pts, conf):
        self.last_cx, self.last_cy = cx, cy
        self.last_pts, self.last_conf = pts, conf
        self.lost_frames = 0

    def mark_lost(self): self.lost_frames += 1

    @property
    def is_active(self):
        return self.last_cx is not None and self.lost_frames <= self.max_lost

tracker = ObjectTracker(max_lost=8)
print("[YOLO] Running — press Q to quit")
t = time.time()

while True:
    ret, frame = cap.read()
    if not ret:
        print("[YOLO] Video ended.")
        break

    try:
        results = model.track(
            frame, conf=CONF, device=device,
            tracker="bytetrack.yaml", persist=True, verbose=False
        )
    except Exception as e:
        # Fallback to non-tracking inference if track fails
        print(f"[WARN] Track failed ({e}), using predict fallback")
        results = model(frame, conf=CONF, device=device, verbose=False)

    r = results[0]
    fps = 1.0 / (time.time() - t); t = time.time()
    detected_this_frame = False

```

Figure 6.23: YOLO Object Detection Code – Part 2

```python
    if r.masks is not None:
        for mask_pts, box in zip(r.masks.xy, r.boxes):
            label = r.names[int(box.cls[0])]
            conf  = float(box.conf[0])
            if TARGET and label.lower() != TARGET.lower():
                continue

            detected_this_frame = True
            pts = mask_pts.astype(int).reshape(-1, 1, 2)

            M = cv2.moments(pts)
            cx = int(M["m10"]/M["m00"]) if M["m00"] != 0 else (int(box.xyxy[0][0])+int(box.xyxy[0][2]))//2
            cy = int(M["m01"]/M["m00"]) if M["m00"] != 0 else (int(box.xyxy[0][1])+int(box.xyxy[0][3]))//2
            tracker.update(cx, cy, pts, conf)

            overlay = frame.copy()
            cv2.fillPoly(overlay, [pts], (0,255,0))
            cv2.addWeighted(overlay, 0.35, frame, 0.65, 0, frame)
            cv2.polylines(frame, [pts], True, (0,255,0), 2)

            x1,y1,x2,y2 = map(int, box.xyxy[0])
            cv2.rectangle(frame, (x1,y1), (x2,y2), (0,200,0), 2)
            tid = f" ID:{int(box.id[0])}" if box.id is not None else ""
            cv2.putText(frame, f"{label}{tid} {conf:.2f}", (x1,y1-10),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0,255,0), 2)
            cv2.circle(frame, (cx,cy), 7, (0,0,255), -1)
            cv2.putText(frame, f"({cx},{cy})", (cx+8,cy),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0,0,255), 2)
            print(f"  >> {label}{tid} | conf={conf:.2f} | centroid=({cx},{cy})")

    if not detected_this_frame:
        tracker.mark_lost()
        if tracker.is_active and tracker.last_pts is not None:
            alpha = 1.0 - (tracker.lost_frames / tracker.max_lost)
            overlay = frame.copy()
            cv2.fillPoly(overlay, [tracker.last_pts], (0,165,255))
            cv2.addWeighted(overlay, 0.25*alpha, frame, 1-0.25*alpha, 0, frame)
            cv2.polylines(frame, [tracker.last_pts], True, (0,165,255), 2)
            cv2.circle(frame, (tracker.last_cx,tracker.last_cy), 7, (0,165,255), -1)
            cv2.putText(frame, f"LAST ({tracker.last_cx},{tracker.last_cy})",
                        (tracker.last_cx+8,tracker.last_cy),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0,165,255), 2)
            cv2.putText(frame, f"Lost:{tracker.lost_frames}/{tracker.max_lost}",
                        (10,90), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0,165,255), 2)

```

Figure 6.24: YOLO Object Detection Code – Part 3

```python

    status = "TRACKING" if detected_this_frame else ("LOST" if tracker.is_active else "SEARCHING")
    color  = (0,255,0) if detected_this_frame else ((0,165,255) if tracker.is_active else (0,0,255))
    cv2.putText(frame, f"FPS:{fps:.1f} | {device.upper()} | {status}",
                (10,30), cv2.FONT_HERSHEY_SIMPLEX, 0.7, color, 2)
    cv2.putText(frame, f"Target:{TARGET or 'all'} | Model:{os.path.basename(MODEL_PATH)}",
                (10,60), cv2.FONT_HERSHEY_SIMPLEX, 0.55, (0,255,255), 2)

    if writer: writer.write(frame)
    cv2.imshow("YOLO11s-seg | Robot Vision", frame)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
if writer:
    writer.release()
    print(f"[YOLO] Saved → {OUTPUT_PATH}")
cv2.destroyAllWindows()
print("[YOLO] Done.")
```

Figure 6.25: YOLO Object Detection Code – Part 4

6.12 Servo Control Code

```
#include <Wire.h>
#include <Adafruit_PWMServoDriver.h>

Adafruit_PWMServoDriver pca = Adafruit_PWMServoDriver(0x40);

const int SERVO_COUNT = 4;
const int SERVO_FREQ  = 50;
const int STEP_SIZE   = 10;
const int UPDATE_MS   = 30;

const int ARM_MIN  = 110;
const int ARM_MAX  = 590;
const int GRIP_MIN = 125;
const int GRIP_MAX = 400;

const int CH_BASE     = 0;
const int CH_SHOULDER = 1;
const int CH_ELBOW    = 2;
const int CH_GRIPPER  = 3;

int current_pwm[SERVO_COUNT];
int target_pwm[SERVO_COUNT];
unsigned long lastUpdate = 0;

int angleToPWM(int angle, int pwmMin, int pwmMax) {
  angle = constrain(angle, 0, 180);
  return pwmMin + ((pwmMax - pwmMin) * angle) / 180;
}

void setArmTarget(int base, int shoulder, int elbow) {
  target_pwm[CH_BASE]     = angleToPWM(base, ARM_MIN, ARM_MAX);
  target_pwm[CH_SHOULDER] = angleToPWM(shoulder, ARM_MIN, ARM_MAX);
  target_pwm[CH_ELBOW]    = angleToPWM(elbow, ARM_MIN, ARM_MAX);
}

void setGripperTarget(int angle) {
  target_pwm[CH_GRIPPER] = angleToPWM(angle, GRIP_MIN, GRIP_MAX);
}
```

Figure 6.26: Servo Control Code – Part 1

```
void updateServos() {
  for (int i = 0; i < SERVO_COUNT; i++) {
    if (current_pwm[i] == target_pwm[i]) continue;

    int error = target_pwm[i] - current_pwm[i];
    int step  = min(STEP_SIZE, abs(error));

    current_pwm[i] += (error > 0) ? step : -step;
    current_pwm[i] = constrain(current_pwm[i], 0, 4095);

    pca.setPWM(i, 0, current_pwm[i]);
  }
}

void processCommand(char* cmd) {
  if (cmd[0] == 'd') {
    int a, b, c;
    if (sscanf(cmd + 1, "%d %d %d", &a, &b, &c) == 3) {
      setArmTarget(a, b, c);
      Serial.println("ok");
    } else {
      Serial.println("err: d <base> <shoulder> <elbow>");
    }

  } else if (cmd[0] == 'g') {
    int g;
    if (sscanf(cmd + 1, "%d", &g) == 1) {
      setGripperTarget(g);
      Serial.println("ok");
    } else {
      Serial.println("err: g <angle>");
    }

  } else if (cmd[0] == '?') {
    Serial.printf("pwm %d %d %d %d\n",
      current_pwm[0], current_pwm[1],
      current_pwm[2], current_pwm[3]);

```

Figure 6.27: Servo Control Code – Part 2

```
  } else {
    Serial.println("err: unknown");
  }
}

char rxBuf[20];
int rxLen = 0;

void readSerial() {
  while (Serial.available()) {
    char c = Serial.read();

    if (c == '\n' || c == '\r') {
      if (rxLen > 0) {
        rxBuf[rxLen] = '\0';
        processCommand(rxBuf);
        rxLen = 0;
      }
    }
    else if (rxLen < sizeof(rxBuf) - 1) {
      rxBuf[rxLen++] = c;
    }
    else {
      rxLen = 0;
    }
  }
}

```

Figure 6.28: Servo Control Code – Part 3

```
void setup() {
  Serial.begin(115200);
  Wire.begin();
  Wire.setClock(400000);

  pca.begin();
  pca.setPWMFreq(SERVO_FREQ);
  delay(10);

  int init[SERVO_COUNT] = { ARM_MIN, 350, 230, GRIP_MIN};

  for (int i = 0; i < SERVO_COUNT; i++) {
    current_pwm[i] = target_pwm[i] = init[i];
    pca.setPWM(i, 0, current_pwm[i]);
  }

  lastUpdate = millis();

  Serial.println("ready");
  Serial.println("cmds: d <b> <s> <e> | g <angle> | ?");
}

void loop() {
  readSerial();

  unsigned long now = millis();
  if (now - lastUpdate >= UPDATE_MS) {
    lastUpdate = now;
    updateServos();
  }
}
```

Figure 6.29: Servo Control Code – Part 4

6.13 TOF Sensor Code

```
#include <Wire.h>
#include <Adafruit_VL53L1X.h>

Adafruit_VL53L1X sensor = Adafruit_VL53L1X();

void setup() {
  Serial.begin(115200);
  Wire.begin(4,5);

  if (!sensor.begin()) {
    Serial.println("Sensor not detected");
    while (1) {
      delay(1000);
    }
  }
  sensor.startRanging();
}

void loop() {
  while (Serial.available()) {
    char c = Serial.read();

    if (c == 's') {
      while(!sensor.dataReady());
      int distance = sensor.distance();
      Serial.println(distance);
      sensor.clearInterrupt();
    }
  }
}
```

Figure 6.30: TOF Sensor Code

Bibliography

[1] A. P. Naik and A. Abraham, “Phoneme-based speech recognition for commanding the robotic arm,” *arXiv preprint arXiv:2001.01222*, 2020.

[2] H. Kumar et al., “Control of robot arm based on speech recognition using MFCC and KNN,” *IEEE*, 2017.

[3] R. V. Dharaskar, S. A. Chhabria, and S. Ganorkar, “Robotic arm control using gesture and voice,” *International Journal of Computer, Information Technology & Bioinformatics (IJCITB)*, vol. 1, no. 1, pp. 41–46, 2012.

[4] R. Aswinbalaji and A. Arunraja, “Wireless voice controlled robotics arm,” *International Journal of Emerging Technology in Computer Science & Electronics (IJETCSE)*, vol. 12, no. 4, pp. 1–6, 2015.

[5] S. Kariuki et al., “Pick and place control of a 3-DOF robot manipulator,” *MDPI Machines*, 2024.

[6] L. P. et al., “Implementation of pick and place robotic arm using speech processing and IoT,” *NCCIP*, 2019.

[7] M. Rashed et al., “Voice controlled object grasping robotic arm for visually impaired disabled veterans,” *IEEE*, 2021.

[8] S. A. et al., *Voice Controlled Camera-Assisted Pick and Place Robot Using Raspberry Pi*, 2020.

[9] S. K. Behera, M. Mohanraj, and P. Prem, “Smart autonomous wireless gripper for human-robot collaborative operation,” *Procedia CIRP*, 2025.

[10] P. Gupta et al., “AI-based multimodal robotic arm for adaptive human-robot interaction,” *MDPI Applied Sciences*, 2024.

[11] K. Ghadge, A. More, S. Gaikwad, and S. Chillal, “Robotic arm for pick and place application,” *International Journal of Mechanical Engineering and Technology*, vol. 9, no. 1, pp. 125–133, 2018.

[12] R. Patel et al., “Voice-controlled pick and place robot,” *IJARSCT*, 2025.

Biodata

Name: Revan MJ

Mobile No.: 7305059695

E-mail: revan.mj2022@vitstudent.ac.in

Permanent Address: B1, Kolavizhi Apartments, 97, Mt Poonamallee High Road, Porur, Chennai - 600116.

Name: Velmurugan S

Mobile No.: 9353679917

E-mail: velmurugan.s2022@vitstudent.ac.in

Permanent Address: No.27, Sapthagiri Colony 1st Street, Jafferkhanpet, Chennai – 600083.

Name: Sham Ganesh M

Mobile No.: 7639971487

E-mail: shamganesh.m2022@vitstudent.ac.in

Permanent Address: 28, Gokul Nagar, Thirumullaivoyal, Chennai – 600062.